

LABORATORY OBSERVATION / RECORD

Course	Full Stack Web Development	Experiment No.	Task 5
Course Code	23CM4121	Student Name	R Abhiram
Reg. No.	A24126552114	Date	21-09-2026

1. TITLE

JavaScript Functions vs Classes – Creating Multiple Objects with Common Properties and Methods using a Car Class

2. AIM

To differentiate between JavaScript functions and classes, and to demonstrate with a **Car** class how one class can be used to create multiple objects that share common properties (brand, model, year, isEngineRunning) and common methods (startEngine() and getDetails()).

3. OBJECTIVE

The objectives of this experiment are:

- To understand what a function and a class are in JavaScript.
- To compare functions and classes on syntax, instantiation, hoisting, methods and inheritance.
- To define a class with a constructor and shared methods.
- To create multiple objects from one class using the **new** keyword.
- To show that every object has its own property values while all objects share the same methods.
- To run the program with Node.js and verify the actual output against the expected output.

4. THEORY

JavaScript function: a reusable block of code that performs a task when it is called. A function can also be used as a constructor (called with **new**) to build objects, but the shared methods must then be attached manually to its prototype. **JavaScript class:** introduced in ES6, a class is a blueprint for creating objects. It keeps the constructor and the methods together in one block. Internally a class is still a special kind of function, so it is called syntactic sugar over JavaScript's prototype-based inheritance.

- **constructor():** special method that runs automatically every time an object is created with **new**; it sets the object's own properties using **this**.
- **Property:** data stored in an object. Every object of the class has the same property names, but each holds its own values.
- **Method:** a function written inside the class body. It is stored once on the class prototype and every object can use it.
- **Object / instance:** a real entity created from the class, e.g. car1 and car2.
- **this:** refers to the object that is being created or that is calling the method.

Differences between JavaScript functions and classes

Point	Function	Class
Purpose	Performs a task or returns a value; can also act as an old-style constructor.	Blueprint for creating many objects with shared properties and methods.
Syntax	function name(params) { ... }	class Name { constructor() { ... } method() { ... } }
Creating objects	Only when written as a constructor function and called with new.	Always created with new Name(...).
Calling without new	Allowed (a normal call; this may be undefined or global).	Not allowed – throws TypeError: Class constructor cannot be invoked without 'new'.
Methods	Attached separately: Fn.prototype.method = function() {...}	Written inside the class body; automatically placed on the prototype.
Hoisting	Function declarations are hoisted and can be called before they are defined.	Cannot be used before the declaration (temporal dead zone – ReferenceError).
Strict mode	Non-strict by default.	Class body always runs in strict mode.
Inheritance	Manual prototype linking (Object.create, call).	Simple with extends and super().
typeof	'function'	'function' (a class is a special function).
Best used for	Small reusable operations, callbacks and utilities.	Modelling real-world entities such as Car, Student or Employee.

How a class creates multiple objects

A class is written once, but `new` can be used any number of times. Each call to `new Car( . . . )` does the following:

- Creates an empty object.
- Links it to `Car.prototype`, so it can use the shared methods `startEngine()` and `getDetails()`.
- Runs the constructor with `this` pointing to the new object, which stores that object's own brand, model, year and `isEngineRunning` (false).
- Returns the finished object, which is saved in a variable (car1, car2).

Common properties: car1 and car2 have the same property names, but their values are different and independent. **Common methods:** `startEngine()` and `getDetails()` are stored only once on `Car.prototype` and shared by all objects, which saves memory and avoids repeating code. The method uses `this` to work on the data of whichever object calls it.

Object structure used in this experiment

```
Car.prototype  (one copy, shared by all objects)
|-- startEngine()
`-- getDetails()
    ^
    | prototype link
+-----+-----+
|               |
car1            car2
|-- brand: "Toyota"    |-- brand: "Tesla"
|-- model: "Corolla"   |-- model: "Model 3"
|-- year: 2021         |-- year: 2024
`-- isEngineRunning: false  `-- isEngineRunning: false
```

Flow: class Car (blueprint) → new Car(...) → constructor sets the object's own properties → object linked to the shared prototype → object.method() runs using that object's data

5. ALGORITHM / PROCEDURE

1. Create a project folder and open a terminal in it (Node.js must be installed).
2. Create the file pbm5.js.
3. Define the class Car using the class keyword.
4. Write constructor(brand, model, year) that stores the three values and sets this.isEngineRunning = false.
5. Add the shared method startEngine() that sets isEngineRunning to true and prints a message.
6. Add the shared method getDetails() that prints the year, brand and model.
7. Create two objects car1 and car2 using new Car(...) with different values.
8. Call getDetails() on both objects, call startEngine() on car1 and print isEngineRunning of both objects.
9. Run the program using node pbm5.js and note the output.
10. Run the additional test cases and verify the actual output against the expected output.

6. PROGRAM

Command used

```
node pbm5.js
```

pbm5.js

```
class Car {
  // 1. Constructor sets up object-specific properties
  constructor(brand, model, year) {
    this.brand = brand;
    this.model = model;
    this.year = year;
    this.isEngineRunning = false;
  }

  // 2. Shared Method (available to all instances)
  startEngine() {
    this.isEngineRunning = true;
    console.log(`The ${this.brand} ${this.model}'s engine is now running.`);
  }

  // Another Shared Method
  getDetails() {
    console.log(`${this.year} ${this.brand} ${this.model}`);
  }
}

// Object 1
const car1 = new Car("Toyota", "Corolla", 2021);

// Object 2
const car2 = new Car("Tesla", "Model 3", 2024);

// Accessing properties (Unique to each object)
console.log(car1.getDetails()); // Output: "2021 Toyota Corolla"
console.log(car2.getDetails()); // Output: "2024 Tesla Model 3"

// Calling shared methods
console.log(car1.startEngine()); // Output: "The Toyota Corolla's engine is now running."

// Unique states
console.log(car1.isEngineRunning); // true
console.log(car2.isEngineRunning); // false (car2's engine was never started)
```

7. CODE EXPLANATION

Code	Explanation
<code>class Car { ... }</code>	Declares a class named Car. It is the blueprint used to create car objects.
<code>constructor(brand, model, year)</code>	Runs automatically each time <code>new Car(...)</code> is called and receives the values for the new object.
<code>this.brand = brand; this.model = model; this.year = year;</code>	<code>this</code> refers to the object being created. Each object gets its own copy of brand, model and year.
<code>this.isEngineRunning = false;</code>	Common property with a default value; every new car starts with the engine off.

Code	Explanation
<code>startEngine() { ... }</code>	Shared method: sets <code>isEngineRunning</code> of the calling object to true and prints a message. It has no return statement.
<code>getDetails() { ... }</code>	Shared method: prints the year, brand and model of the calling object. It also returns nothing.
<code>const car1 = new Car("Toyota", "Corolla", 2021);</code>	<code>new</code> creates an empty object, links it to <code>Car.prototype</code> , runs the constructor and returns the object (Object 1).
<code>const car2 = new Car("Tesla", "Model 3", 2024);</code>	A second, independent object created from the same class (Object 2).
<code>console.log(car1.getDetails());</code>	<code>getDetails()</code> prints the details itself and returns undefined, so <code>console.log</code> then prints undefined.
<code>console.log(car1.startEngine());</code>	<code>startEngine()</code> prints the running message, then <code>console.log</code> prints undefined (the return value).
<code>console.log(car1.isEngineRunning);</code>	Reads the property directly: true, because <code>car1</code> 's engine was started.
<code>console.log(car2.isEngineRunning);</code>	false, because <code>car2</code> 's engine was never started; the state of each object is separate.

8. TEST CASES AND EXECUTION DATA

TC-01 was run on the student's system (screenshot in Section 9). TC-02 to TC-08 were run with Node.js using the Car class of pbm5.js followed by the test lines shown in Section 9. In each case the printed output was compared with the expected result.

Test Case	Action	Property Checked	Expected Result	Status
TC-01	node pbm5.js	Output of all console.log lines	8 lines: details, undefined, details, undefined, message, undefined, true, false	Pass
TC-02	Create car1 and read its properties	brand, model, year, isEngineRunning of car1	Toyota, Corolla, 2021, false	Pass
TC-03	Call car1.startEngine(); compare car1 and car2	isEngineRunning of car1 and car2	car1: true, car2: false	Pass
TC-04	Compare the methods of car1 and car2	startEngine and getDetails on the prototype	Same function (true); not own property (false); on Car.prototype (true)	Pass
TC-05	Store the value returned by getDetails()	Return value of getDetails()	Details printed; return value undefined	Pass
TC-06	Check typeof, instanceof and Object.keys()	Type of Car; instance check; own properties	function, true, four data properties only	Pass
TC-07	Call Car(...) without new	Error raised	TypeError: Class constructor Car cannot be invoked without 'new'	Pass
TC-08	Repeat the logic with a function + prototype	Output and shared method	Same output; shared method (true)	Pass

9. OUTPUT

Output 1: TC-01 – Running pbm5.js (two Car objects, shared methods)

TC-01 – node pbm5.js (run by student)

PASS

```
PS C:\Users\varsh\OneDrive\Desktop\114.html> node pbm5.js
2021 Toyota Corolla
undefined
2024 Tesla Model 3
undefined
The Toyota Corolla's engine is now running.
undefined
true
false
PS C:\Users\varsh\OneDrive\Desktop\114.html>
```

Property checked	Expected	Actual (read from run)	Result
line 1 (car1.getDetails())	2021 Toyota Corolla	2021 Toyota Corolla	PASS
line 2 (return value)	undefined	undefined	PASS
line 3 (car2.getDetails())	2024 Tesla Model 3	2024 Tesla Model 3	PASS
line 4 (return value)	undefined	undefined	PASS
line 5 (car1.startEngine())	The Toyota Corolla's engine is now running.	The Toyota Corolla's engine is now running.	PASS
line 6 (return value)	undefined	undefined	PASS
car1.isEngineRunning	true	true	PASS
car2.isEngineRunning	false	false	PASS

Each `getDetails()` and `startEngine()` call prints its own text and then `console.log` prints the returned value, `undefined`. The last two lines show that only `car1`'s engine state changed.

Test case outputs (executed with Node.js)

Output 2: TC-02 – Object created with its own properties

TC-02 – Object created with its own properties

PASS

Test code (added after the Car class):

```
const car1 = new Car("Toyota", "Corolla", 2021);
console.log(car1.brand);
console.log(car1.model);
console.log(car1.year);
console.log(car1.isEngineRunning);
console.log(JSON.stringify(car1));
```

```
$ node tc02.js
Toyota
Corolla
2021
false
{"brand":"Toyota","model":"Corolla","year":2021,"isEngineRunning":false}
```

Property checked	Expected	Actual (read from run)	Result
car1.brand	Toyota	Toyota	PASS
car1.model	Corolla	Corolla	PASS
car1.year	2021	2021	PASS
car1.isEngineRunning	false	false	PASS
object data (JSON)	{"brand":"Toyota","model":"Corolla","year":2021,"isEngineRunning":false}	{"brand":"Toyota","model":"Corolla","year":2021,"isEngineRunning":false}	PASS

car1 holds its own brand, model and year, and isEngineRunning starts as false (set by the constructor for every object).

Output 3: TC-03 – Each object keeps its own state

TC-03 – Each object keeps its own state

PASS

Test code (added after the Car class):

```
const car1 = new Car("Toyota", "Corolla", 2021);
const car2 = new Car("Tesla", "Model 3", 2024);
car1.startEngine();
console.log(car1.isEngineRunning);
console.log(car2.isEngineRunning);
```

```
$ node tc03.js
The Toyota Corolla's engine is now running.
true
false
```

Property checked	Expected	Actual (read from run)	Result
startEngine() message	The Toyota Corolla's engine is now running.	The Toyota Corolla's engine is now running.	PASS
car1.isEngineRunning (after)	true	true	PASS
car2.isEngineRunning (after)	false	false	PASS

Starting *car1*'s engine changed only *car1*; *car2* kept its own value (*false*).

Output 4: TC-04 – Methods are shared by all objects

TC-04 – Methods are shared by all objects

PASS

Test code (added after the Car class):

```
const car1 = new Car("Toyota", "Corolla", 2021);
const car2 = new Car("Tesla", "Model 3", 2024);
console.log(car1.startEngine === car2.startEngine);
console.log(car1.getDetails === car2.getDetails);
console.log(car1.hasOwnProperty("startEngine"));
console.log(Car.prototype.hasOwnProperty("startEngine"));
```

```
$ node tc04.js
true
true
false
true
```

Property checked	Expected	Actual (read from run)	Result
startEngine same in car1 & car2	true	true	PASS
getDetails same in car1 & car2	true	true	PASS
startEngine own property of car1	false	false	PASS
startEngine on Car.prototype	true	true	PASS

The methods are stored once on *Car.prototype* and all objects use that single copy.

Output 5: TC-05 – Methods print and return undefined

TC-05 – Methods print and return undefined

PASS

Test code (added after the Car class):

```
const car1 = new Car("Toyota", "Corolla", 2021);
const car2 = new Car("Tesla", "Model 3", 2024);
const r = car2.getDetails();
console.log(r);
console.log(r === undefined);
```

```
$ node tc05.js
2024 Tesla Model 3
undefined
true
```

Property checked	Expected	Actual (read from run)	Result
text printed inside getDetails()	2024 Tesla Model 3	2024 Tesla Model 3	PASS
console.log(returned value)	undefined	undefined	PASS
returned value === undefined	true	true	PASS

getDetails() prints its text itself but has no return statement, so it returns undefined. This is why 'undefined' appears in the output of pbm5.js.

Output 6: TC-06 – Class type and own properties

TC-06 – Class type and own properties

PASS

Test code (added after the Car class):

```
const car1 = new Car("Toyota", "Corolla", 2021);
console.log(typeof Car);
console.log(car1 instanceof Car);
console.log(JSON.stringify(Object.keys(car1)));
```

```
$ node tc06.js
function
true
["brand","model","year","isEngineRunning"]
```

Property checked	Expected	Actual (read from run)	Result
typeof Car	function	function	PASS
car1 instanceof Car	true	true	PASS
Object.keys(car1)	["brand","model","year","isEngineRunning"]	["brand","model","year","isEngineRunning"]	PASS

A class is a special function; the object is an instance of Car and owns only the data properties, not the methods.

Output 7: TC-07 – Class cannot be called without new

TC-07 – Class cannot be called without new

PASS

Test code (added after the Car class):

```
try {
  const c = Car("Honda", "Civic", 2020);
} catch (e) {
  console.log(e.name + ": " + e.message);
}
```

```
$ node tc07.js
TypeError: Class constructor Car cannot be invoked without 'new'
```

Property checked	Expected	Actual (read from run)	Result
error thrown	TypeError: Class constructor Car cannot be invoked without 'new'	TypeError: Class constructor Car cannot be invoked without 'new'	PASS

Unlike an ordinary function, a class cannot be called without new.

Output 8: TC-08 – Same idea using a constructor function

TC-08 – Same idea using a constructor function

PASS

Test code (tc08.js):

```
// Same idea written as a constructor function (pre-ES6 style)
function CarFn(brand, model, year) {
  this.brand = brand;
  this.model = model;
  this.year = year;
  this.isEngineRunning = false;
}
CarFn.prototype.startEngine = function () {
  this.isEngineRunning = true;
  console.log(`The ${this.brand} ${this.model}'s engine is now running.`);
};
CarFn.prototype.getDetails = function () {
  console.log(`${this.year} ${this.brand} ${this.model}`);
};
const a = new CarFn("Toyota", "Corolla", 2021);
const b = new CarFn("Tesla", "Model 3", 2024);
a.getDetails();
b.getDetails();
a.startEngine();
console.log(a.startEngine === b.startEngine);
```

```
$ node tc08.js
2021 Toyota Corolla
2024 Tesla Model 3
The Toyota Corolla's engine is now running.
true
```

Property checked	Expected	Actual (read from run)	Result
a.getDetails()	2021 Toyota Corolla	2021 Toyota Corolla	PASS
b.getDetails()	2024 Tesla Model 3	2024 Tesla Model 3	PASS
a.startEngine()	The Toyota Corolla's engine is now running.	The Toyota Corolla's engine is now running.	PASS
a.startEngine === b.startEngine	true	true	PASS

A constructor function with prototype methods gives the same result, but a class keeps the constructor and methods together in one block.

10. RESULT

Thus, the difference between JavaScript functions and classes was studied, and a Car class with a constructor and two shared methods was created. Two objects (car1 and car2) were created from the same class; each object holds its own property values and state, while both share the same methods. The program output and the results of eight test cases confirmed that the program behaved as expected.